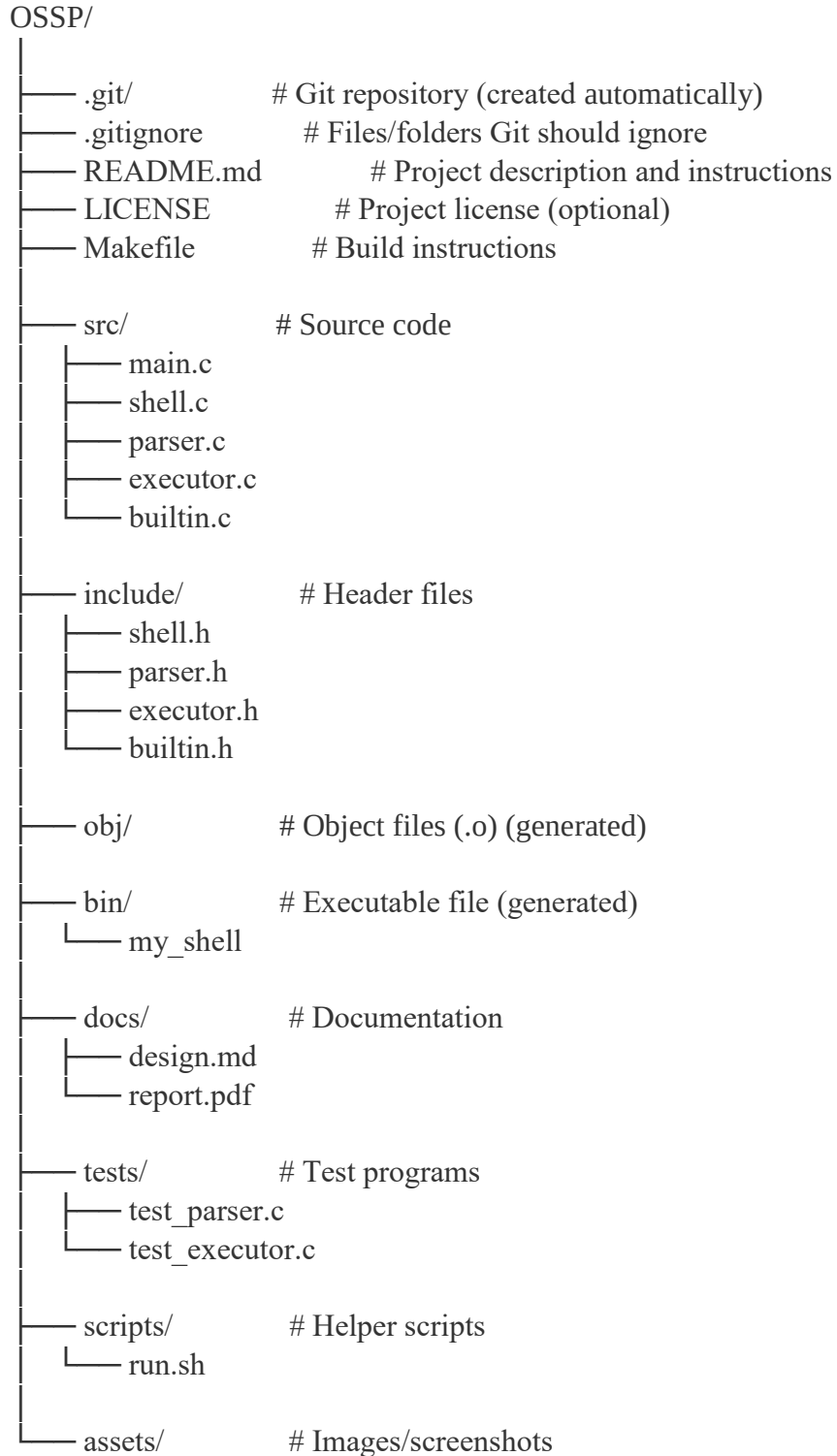


1. To Install Linux VM, Configure GCC, Setup Git Repository, Create Project Structure, Understand Shell Architecture, Build Initial Makefile.

Task -1: Create Project Structure as follows:



└─ architecture.png

Note: For creating this structure use mkdir and touch commands.

```
sindhuja@SINDHUJA: ~/OSSF × + ▾
sindhuja@SINDHUJA:~$ cd OSSP
sindhuja@SINDHUJA:~/OSSP$ tree
.
├── LICENSE
├── Makefile
├── README.md
├── assets
│   └── architecture.png
├── bin
│   └── my_shell
├── docs
│   ├── design.md
│   └── report.pdf
├── include
│   ├── builtin.h
│   ├── executor.h
│   ├── parser.h
│   └── shell.h
├── obj
├── scripts
│   └── run.sh
├── src
│   ├── builtin.c
│   ├── executor.c
│   ├── main.c
│   ├── parser.c
│   └── shell.c
└── tests
    ├── test_executor.c
    └── test_parser.c

9 directories, 19 files
sindhuja@SINDHUJA:~/OSSP$ |
```

2. To Analyze process abstraction, execute fork(), understand exec() family, analyze parent-child relationships, inspect process tree, practice system call tracing.

Task-1: Using a manual command understand the fork() and exec() system calls

Task-2: Write a C program to create a new process and accepting the user commands using fork() and exec() system calls.

Example:

```
#include <stdio.h>

#include <unistd.h>

int main() {
    pid_t pid;

    pid = fork(); // Create a new process
    if (pid < 0) {
        printf("Fork failed!\n");
    }
    else if (pid == 0) {
        // Child Process
        printf("This is the Child Process.\n");
        printf("Child PID = %d\n", getpid());
    }
    else {
        // Parent Process
        printf("This is the Parent Process.\n");
        printf("Parent PID = %d\n", getpid());
        printf("Child PID = %d\n", pid);
    }
    return 0;
}
```

sindhuja@SINDHUJA: ~

GNU nano 7.2fork_exec.c

```
#include <stdio.h>
#include <unistd.h>

int main() {
    pid_t pid;

    pid = fork(); // Create a new process

    if (pid < 0) {
        printf("Fork failed!\n");
    }
    else if (pid == 0) {
        // Child Process
        printf("This is the Child Process.\n");
        printf("Child PID = %d\n", getpid());
    }
    else {
        // Parent Process
        printf("This is the Parent Process.\n");
        printf("Parent PID = %d\n", getpid());
        printf("Child PID = %d\n", pid);
    }

    return 0;
}
```

u sindhuja (sindhujaa06@outlook.com) is signed in

^G Help

^X Exit

^O Write Out

^R Read File

^W Where Is

^_ Replace

^K Cut

^U Paste

^T Execute

^J Justify

```
sindhuja@SINDHUJA: ~  
sindhuja@SINDHUJA:~$ ls  
OSSP  fork_exec  fork_exec.c  
sindhuja@SINDHUJA:~$ gcc fork_exec.c -o fork_exec  
sindhuja@SINDHUJA:~$ ./fork_exec  
This is the Parent Process.  
Parent PID = 2617  
Child PID = 2618  
This is the Child Process.  
Child PID = 2618  
sindhuja@SINDHUJA:~$ |
```